

ASSIGNMENT – 09

NAME : M.AKSHAYPATEL

HALL TICKET NO : 2303A51745

BATCH : 11

Question :

Expected Time to complete 1 Interacting with DeFi Protocols (Testnet) Objective To interact with a decentralized finance (DeFi) protocol on a blockchain testnet by writing and executing a smart contract that deposits and withdraws tokens, demonstrating basic DeFi operations without real funds.

Requirements

- Install Node.js and npm
- Install VS Code
- Install VS Code extensions: o Solidity o Hardhat
- Use a testnet (Sepolia / Goerli / Mumbai) •

Create a smart contract and a script to interact with a DeFi protocol (mock lending contract) • Use functions for deposit and withdrawal

- Test interactions using test tokens • Add inline comments explaining the logic

Tools & Technologies

- Solidity
- Hardhat
- JavaScript (Node.js)
- Ethereum Testnet (Sepolia)
- MetaMask (Testnet account) Practical Description In this practical, we simulate interaction with a DeFi protocol by creating a simple lending contract where:
 - Users can deposit ETH
 - Users can withdraw ETH
 - Contract tracks balances internally This represents how DeFi protocols like Aave or Compound work on testnets.

CODE :

```
import tkinter as tk from tkinter
```

```
import messagebox class
```

```
DeFiLending: def
```

```

__init__(self):    self.balances
= {}

    def deposit(self, user, amount):
if amount <= 0:
        return "Deposit must be greater than 0"
if user not in self.balances:
self.balances[user] = 0    self.balances[user]
+= amount    return f"{user} deposited
{amount} ETH"    def withdraw(self, user,
amount):    if user not in self.balances:
return "User does not exist"    if amount <=
0:
        return "Withdrawal must be greater than
0"    if self.balances[user] < amount:
return "Insufficient balance"
self.balances[user] -= amount    return f"{user}
withdrew {amount} ETH"    def get_balance(self,
user):
        return self.balances.get(user, 0)
class DeFiApp:
    def __init__(self, root):
        self.contract = DeFiLending()    self.root = root    self.root.title("DeFi
Lending Protocol Simulator")    self.root.geometry("400x350")
tk.Label(root, text="DeFi Simulator", font=("Arial", 16, "bold")).pack(pady=10)
tk.Label(root, text="User Name").pack()    self.user_entry = tk.Entry(root)
self.user_entry.pack()    tk.Label(root, text="Amount (ETH)").pack()
self.amount_entry = tk.Entry(root)

        self.amount_entry.pack()    tk.Button(root, text="Deposit",
command=self.deposit).pack(pady=5)    tk.Button(root, text="Withdraw",
command=self.withdraw).pack(pady=5)    tk.Button(root, text="Check Balance",

```

```

command=self.check_balance).pack(pady=5)    self.output = tk.Label(root, text="",
fg="blue")    self.output.pack(pady=10)    def deposit(self):
    user = self.user_entry.get()
    try:
        amount = float(self.amount_entry.get())
    result = self.contract.deposit(user, amount)
    self.output.config(text=result)    except:
        messagebox.showerror("Error", "Enter valid amount")
def withdraw(self):
    user = self.user_entry.get()
    try:
        amount = float(self.amount_entry.get())
    result = self.contract.withdraw(user, amount)
    self.output.config(text=result)    except:
        messagebox.showerror("Error", "Enter valid amount")

def check_balance(self):
    user = self.user_entry.get()    balance =
self.contract.get_balance(user)
self.output.config(text=f"{user} Balance: {balance} ETH")
if __name__ == "__main__":
    root = tk.Tk()
    app = DeFiApp(root)
    root.mainloop() IMPLEMENTATION :

```

```

index.html 9+  gui.py
C: > Users > Rushi > Downloads > gui.py > DeFiApp > check_balance
1  import tkinter as tk
2  from tkinter import messagebox
3  class DeFiLending:
4      def __init__(self):
5          self.balances = {}
6      def deposit(self, user, amount):
7          if amount <= 0:
8              return "Deposit must be greater than 0"
9          if user not in self.balances:
10             self.balances[user] = 0
11             self.balances[user] += amount
12             return f"{user} deposited {amount} ETH"
13      def withdraw(self, user, amount):
14          if user not in self.balances:
15              return "User does not exist"
16          if amount <= 0:
17              return "Withdrawal must be greater than 0"
18          if self.balances[user] < amount:
19              return "Insufficient balance"
20          self.balances[user] -= amount
21          return f"{user} withdrew {amount} ETH"
22      def get_balance(self, user):
23          return self.balances.get(user, 0)
24  class DeFiApp:
25      def __init__(self, root):
26          self.contract = DeFiLending()
27          self.root = root
28          self.root.title("DeFi Lending Protocol Simulator")
29          self.root.geometry("400x350")
30          tk.Label(root, text="DeFi Simulator", font=("Arial", 16, "bold")).pack(pady=10)
31          tk.Label(root, text="User Name").pack()
32          self.user_entry = tk.Entry(root)
33          self.user_entry.pack()

```

```

C: > Users > Rushi > Downloads > gui.py > DeFiApp > check_balance
24  class DeFiApp:
25      def __init__(self, root):
38          tk.Button(root, text="Withdraw", command=self.withdraw).pack(pady=5)
39          tk.Button(root, text="Check Balance", command=self.check_balance).pack(pady=5)
40          self.output = tk.Label(root, text="", fg="blue")
41          self.output.pack(pady=10)
42      def deposit(self):
43          user = self.user_entry.get()
44          try:
45              amount = float(self.amount_entry.get())
46              result = self.contract.deposit(user, amount)
47              self.output.config(text=result)
48          except:
49              messagebox.showerror("Error", "Enter valid amount")
50      def withdraw(self):
51          user = self.user_entry.get()
52          try:
53              amount = float(self.amount_entry.get())
54              result = self.contract.withdraw(user, amount)
55              self.output.config(text=result)
56          except:
57              messagebox.showerror("Error", "Enter valid amount")
58
59      def check_balance(self):
60          user = self.user_entry.get()
61          balance = self.contract.get_balance(user)
62          self.output.config(text=f"{user} Balance: {balance} ETH")
63  if __name__ == "__main__":
64      root = tk.Tk()
65      app = DeFiApp(root)
66      root.mainloop()

```

OUTPUT :

DeFi Lending Protocol Simulator

DeFi Simulator

User Name

abc

Amount (ETH)

1000

Deposit

Withdraw

Check Balance

abc Balance: 1100.0 ETH